
cidrtools-cffi API Reference

Release 1.4.2

Gene C

Oct 06, 2026

PYTHON API REFERENCE:

1	cidrtools-cffi	1
1.1	Overview	1
1.2	Features	1
1.3	Recent Changes	2
1.4	Performance Benchmarks	2
1.5	Installation	2
1.6	Usage Example	3
1.7	API Reference	3
1.8	AI Tooling	4
2	API Reference	5
2.1	class cidrtools	5
3	Appendix	11
3.1	Installation	11
3.2	Dependencies	11
3.3	Philosophy	11
	Python Module Index	13
	Index	15

CIDRTOOLS-CFFI

1.1 Overview

Python CFFI bindings for the high-performance `cidrtools` C library.

This module provides a suite of tools to examine and manipulate network cidr blocks. It has the same functionality as the `cidrtools` library.

The source is available in [Github `cidrtools-cffi`](#) as well as [Arch AUR](#)

This package exposes the complete `cidrtools` API. It provides an fast bridge to the underlying C structures and all the functions in the shared library.

1.1.1 Documentation

The manual provides detailed information and is available in both HTML and PDF formats. Both are installed under `/usr/share/cidrtools/docs`.

The manual is also available at: [readthedocs](#).

1.2 Features

- Wraps C-library into 3 Python classes.
 - `CidrBlock` - for a single cidr.
 - `CidrBlocks` - for collection of cidrs.
 - `CidrTools` - convenient DNS lookup (forward and reverse).
- Optimized performance
 - Runs complex operations (such as block compacting) at (almost) native C speeds.
- Maintainable
 - Written in pure Python with CFFI ABI mode — no compilation needed.
- Modern Tooling
 - Managed using by `uv`, `meson`, and `meson-python`.
- Robust Interface
 - Memory handling / garbage collection

The python module is named *cidrtools* matching the underlying C-library.

1.3 Recent Changes

1.4.1

- Drop unused uv/pyproject.toml (meson does it all)
- Add .gitattributes

1.4.0

- Sync with cidrtools 1.4.0
New `ct_str_to_cidr_block_limit()`, `ct_is_ipv4_or_ipv6()`

1.4 Performance Benchmarks

Below benchmarks the execution time to process a random sample of *100,000 subnets* and compact them down to *54,941 subnets*.

This is significantly faster than a pure Python version built using *ipadress*.

Version Used	Seconds	Relative
-----	-----	-----
(a) Raw C-Code	0.011892861	1.0 x
(b) CFFI Wrapper	0.011995342	1.0 x
(c) Cython/C Bridge	0.021802806	1.8 x
(d) Pure Cython	0.026872699	2.3 x
(e) Pure Python	0.935538276	78.7 x

*100,000 subnets compacted to 54,941 subnets

1.5 Installation

The package is available in the Arch linux AUR using the PKGBUILD provided in the *packaging* directory. It can also be used locally directly from the git repo.

It can also be installed locally using the the provided scripts:

```
./scripts/do-build
./scripts/do-install
./scripts/run-tests
```

The default install directory is *build/pkg*, but *do-install* takes the destination directory as an argument as well.

1.6 Usage Example

1.6.1 High-Level API

```
from cidrtools import CidrBlock, CidrBlocks

# Instantiating one cidr block from a string.
cidr = CidrBlock("192.168.1.0/24")
print(f"IPs within network: {cidr.num_ips}")

# Check if ip or cidr is in the cidr subnet.
ip_str = "192.168.1.50"
is_in = "is" if (ip_str in cidr) else "is not"
print(f' {ip_str} {is_in} in {cidr}')
```

Similarly

```
subcidr = CidrBlock("192.168.1.50")
subcidr in cidr
=> True

=> 192.168.1.50 is in 192.168.1.0/24
```

```
cidrs = CidrBlocks(["192.168.1.0/23", "10.0.0.0/24"])
is_subnet = "is" if cidr.is_subnet_of(cidrs) else "is not"
print(f' {str(cidr)} {is_subnet} a subnet of {list(cidrs)}')
```

Similarly

```
cidr = CidrBlock("192.168.1.0/24")
cidr.is_subnet_of(cidrs)
=> 192.168.1.0/24 is a subnet of ['192.168.0.0/23', '10.0.0.0/24']
```

```
cidrs.add('10.0.2.0/24')
list(cidrs)

=> ['192.168.0.0/23', '10.0.0.0/24', '10.0.2.0/24']

cidrs.compact()
list(cidrs)

=> ['10.0.0.0/24', '10.0.2.0/24', '192.168.0.0/23']
```

```
cidrs = CidrCollection(["192.168.1.0/25", "192.168.1.128/25"])
cidrs.sort()
cidrs.compact()

# back to a standard Python list of strings
print(list(cidrs)) # Output: ['192.168.1.0/24']
```

1.7 API Reference

Documentation is built with Sphinx. The PDF manual is in `src/data/docs` and installed to `/usr/share/cidrtools-cffi`.

To create a local copy of the html and PDF documentation:

```
cd src/data/docs
./build-manual.sh
```

1.8 AI Tooling

Assisted by:

- Anthropic's Claude
- Google's Gemini

API REFERENCE

This section contains the API documentation for the `cidrtools` package.

2.1 class `cidrtools`

`cidrtools-cffi` provides the `cidrtools` module using the fast C-code `cidrtools` library:

- public class/member functions
- wrapper function for each `cidrtool` `ct_xx` function.

class `cidrtools.CidrBlock`(*cidr_str: str*)

Bases: `object`

Class to manage one cidr block. Uses `cidrtools` C-library `CtCidrBlock` structure.

Handles memory allocation, type inspections, range mapping, and contains (is-subnet).

Built on the `cidrtools` library.

`cidr_parts()` → `tuple[str, int]`

Returns the IP address and prefix as a tuple.

Returns

The tuple (ip_address, prefix)

`clean()` → `bool`

Clean up invalid cidr blocks (prefix, host bits being set)

Returns

True on success otherwise False

`fix_host_bits()` → `int`

Cleans up cidr with host bits set, by setting all host bits to zero.

`format_host_bits()` → `str`

Returns a formatted “address” string of the host bits Identical to `get_host_bits()`.

get_host_bits() → str

Extracts the underlying address portion - these are the bits past the network bits defined by the prefix length.

<prefix length of network-bits ... ><host-bits>

E.g. 10.2.2.3/24 => 0.0.0.3 Note that the host bits are tracked internally but the cidr is always “fixed” Example: - cidr = CidrBlock(“192.168.1.22/24”) - str(cidr) -> “192.168.1.0/24” - cidr.get_host_bits() -> “0.0.0.22”

Returns

A string with the network bits set to 0, showing only the host bits.

increment_by(steps: int) → str

Find a new network address advanced by ‘steps’ IP addresses.

Parameters

steps – The number of steps to add to the IP address

Returns

The new address.

property is_ipv4: bool

Returns True if the cidr block is an IPv4 network family

Returns

True if the network family is IPv4, False otherwise.

property is_ipv4_or_ipv6: bool

Returns True if the cidr block is an IPv6 network family.

Returns

True if the network family is IPv6, False otherwise.

property is_ipv6: bool

Returns True if the cidr block is an IPv6 network family.

Returns

True if the network family is IPv6, False otherwise.

is_subnet_of(cidrs: CidrBlocks) → bool

Checks if this block is a subnet of any of the collection of cidrs.

Parameters

cidrs – The collection of cidr blocks to check

Returns

True if is a subnet else False.

property num_ips: int

Returns the total number of IPs inside this cidr block.

Note that for IPv6 this is capped at 64 bit integer max.

Returns

Number of IPs in the cidr block.

set_prefix(*prefix: int*) → int

Modifies the network prefix to be the new value.

Parameters

prefix – The network prefix

Returns

The new prefix value

split(*prefix: int*) → *CidrBlocks*

Splits the cidr block into a collection of smaller subnets using the provided prefix.

Parameters

prefix – The prefix to use to split this cidr.

Returns

The collection of subnets that aggregate to this cidr block.

to_range() → tuple[str, str]

Computes the first and last IP addresses of this cidr block. See also to_range_mid()

Returns

A tuple of (first, last) last IP addresses.

to_range_mid() → tuple[str, str, str]

Compute the first, middle and last IP addresses of this cidr block.

Returns

A tuple of (first, middle, last) IP addresses.

to_string() → str

Return the cidr string. Same as str(self)

Returns

The string representation of the cidr.

class cidrtools.**CidrBlocks**(*initial_cidrs: list[str] | None = None*)

Bases: object

An ordered list of CtCidrs

A list of cidr blocks. Supports a variety of operations via the member functions such as: - compacting - filtering (excluding), - slicing, - cleaning.

Built on the cidrtools library. Manages memory allocations.

add(*cidr: CidrBlock | str*) → bool

Appends a CIDR block to the collection.

Parameters

cidr – The cidr block (object or raw string) to add.

Returns

True if the append succeeded else false.

clean() → bool

Clean any invalid cidr blocks (prefix, host bits being set)

Returns

True if success else False

compact() → int

Compress the list of cidrs to the minimal number of subnets.

This combines adjacent or overlapping subnets to make the smallest number of subnets. This is done in place.

Returns

Return status code from the underlying C function.

exclude(*excluded_cidrs*: [CidrBlocks](#)) → int

Exclude a set of cidr blocks. This is done place.

new cidrs => cidrs - excluded_cidrs.

Parameters

excluded_cidrs – The cidrs to be excluded.

Returns

Return status code of the underlying C library.

intersection(*cidrs*: [CidrBlocks](#), *intersection*: [CidrBlocks](#)) → int

Computes the intersecting subnets with another set of cidrs.

Parameters

cidrs – The subnets to use to get the intersection.

Paramm intersection

The resulting intersecting subnets.

Returns

the integer return code from c-library.

sort() → int

In-place sort of the cidr blocks.

Returns

Return status code from the underlying C library

split_by_family() → tuple[[CidrBlocks](#), [CidrBlocks](#)]

Splits the cidr blocks into separate IPv4 and IPv6 lists.

Returns

tuple of cidr lists - (IPv4_Pool, IPv6_Pool).

Raises

RuntimeError – If errors found by C-library

to_strings() → list[str]

Returns a list of cidr strings. Optimized for speed.

Returns

list of all cidr blocks.

class cidrtools.CidrTools

Bases: object

Class with some additional member functions of the *cidrtools* library.

Underlying cidrtools version and some DNS convenience functions.

static **ct_version**() → str

Returns the native C library version target compilation string.

Returns

The loaded library system release semantic tag value.

Return type

str

static **hostname_to_addresses**(*hostname: str*) → *CidrBlocks*

Resolves a hostname string into a structured collection of CIDR blocks.

Parameters

hostname (*str*) – The explicit domain URL path target string to resolve.

Returns

A fresh collection array tracking resolved system addresses blocks.

Return type

CidrBlocks

Raises

RuntimeError – If standard resolution paths fail or return unexpected status flags.

static **ip_str_to_hostname**(*ip_str: str*) → str

Performs a reverse DNS lookup mapping an active IP address string back to a hostname.

Parameters

ip_str (*str*) – The textual raw representation layout of the target lookup address.

Returns

The fully qualified resolved host identifier name string.

Return type

str

Raises

RuntimeError – If reverse lookup actions time out or tracking channels fail.

static range_to_cidrs(*first_ip: str, last_ip: str*) → *CidrBlocks*

Converts an IP range to a minimal collection of CIDR blocks.

Should this be moved to `CidrBlocks.range_to_cidrs()` ?

Parameters

- **first_ip** – The first network IP of the range
- **last_ip** – The last IP of the range

Returns

A minimal representation tracking collection of matched network blocks.

Raises

- **ValueError** – If input strings map incorrectly to standard protocol types.
- **RuntimeError** – If underlying C matrix calculations encounter configuration faults.

3.1 Installation

On Arch you can build using the provided PKGBUILD in the packaging directory or from the AUR. To build manually, clone the repo and :

```
rm -f dist/*  
./scripts/do-build  
./scripts/do-install [destination]
```

The default destination is *build/pkg*.

3.2 Dependencies

Run Time :

- cidrtools
- python-cffi
- python (3.14 or later)

Building Package :

- git
- meson
- meson-python
- uv
- python-uv-build
- rsync
- python-pytest
- python-pytest-asyncio

3.3 Philosophy

We follow the *live at head commit* philosophy as recommended by Google's Abseil team¹. This means we recommend using the latest commit on git master branch.

¹ <https://abseil.io/about/philosophy#upgrade-support>

PYTHON MODULE INDEX

C

`cidrtools`, 5

INDEX

A

`add()` (*cidrtools.CidrBlocks method*), 6

C

`cidr_parts()` (*cidrtools.CidrBlock method*), 5

`CidrBlock` (class in *cidrtools*), 5

`CidrBlocks` (class in *cidrtools*), 6

`cidrtools`

 module, 5

`CidrTools` (class in *cidrtools*), 7

`clean()` (*cidrtools.CidrBlock method*), 5

`clean()` (*cidrtools.CidrBlocks method*), 7

`compact()` (*cidrtools.CidrBlocks method*), 7

`ct_version()` (*cidrtools.CidrTools static method*), 8

E

`exclude()` (*cidrtools.CidrBlocks method*), 7

F

`fix_host_bits()` (*cidrtools.CidrBlock method*), 5

`format_host_bits()` (*cidrtools.CidrBlock method*), 5

G

`get_host_bits()` (*cidrtools.CidrBlock method*), 5

H

`hostname_to_addresses()` (*cidrtools.CidrTools static method*), 8

I

`increment_by()` (*cidrtools.CidrBlock method*), 5

`intersection()` (*cidrtools.CidrBlocks method*), 7

`ip_str_to_hostname()` (*cidrtools.CidrTools static method*), 8

`is_ipv4` (*cidrtools.CidrBlock property*), 6

`is_ipv4_or_ipv6` (*cidrtools.CidrBlock property*), 6

`is_ipv6` (*cidrtools.CidrBlock property*), 6

`is_subnet_of()` (*cidrtools.CidrBlock method*), 6

M

module

cidrtools, 5

N

`num_ips` (*cidrtools.CidrBlock property*), 6

R

`range_to_cidrs()` (*cidrtools.CidrTools static method*), 8

S

`set_prefix()` (*cidrtools.CidrBlock method*), 6

`sort()` (*cidrtools.CidrBlocks method*), 7

`split()` (*cidrtools.CidrBlock method*), 6

`split_by_family()` (*cidrtools.CidrBlocks method*), 7

T

`to_range()` (*cidrtools.CidrBlock method*), 6

`to_range_mid()` (*cidrtools.CidrBlock method*), 6

`to_string()` (*cidrtools.CidrBlock method*), 6

`to_strings()` (*cidrtools.CidrBlocks method*), 7